

# GramSetu AI - Privacy & Safety Policy

This document describes the privacy guarantees, data storage boundaries, camera permissions, and risk mitigation strategies implemented in the GramSetu AI platform.

## 1. Local-First Privacy Guarantees

GramSetu AI is designed to protect user privacy by executing all core services—including image processing, OCR, and computer vision model inference—locally on the client device.

- **Zero Media Uploads:** Camera snapshots and local file uploads are loaded directly into browser memory buffers. No image files or raw pixels are sent over the network to cloud servers.
- **Off-grid Reliability:** The application remains functional without internet connectivity once static assets are loaded, ensuring that data is not leaked in transit.
- **Local Storage Boundaries:** User history, checklists, and configurations are stored in the browser's **localStorage** sandbox, which is isolated to the application origin.

## 2. Browser Permissions & Hardware Access

The application requests hardware access solely to perform user-initiated tasks:

### Camera Access

- **Usage:** Accessed in the SwachhAudit and JalDrishti modules.
- **Mechanism:** Calls **navigator.mediaDevices.getUserMedia** to stream video frames to local **<video>** elements.
- **Scope:** Video streams are processed locally in memory. No stream data is transmitted or saved outside the browser profile.

### Storage Read Access

- **Usage:** Accessed when a user selects local file uploads in SwachhAudit or GramLipi.
- **Scope:** Files are read in-memory using the browser's **FileReader** API.

## 3. Threat Model & Security Boundaries

### Threat 1: Interception of Sensitive Documents or Snapshots

- **Risk:** Scanned government documents or sanitation photos could be intercepted in transit.
- **Mitigation:** All text extraction (Tesseract OCR) and image processing run locally on the device, eliminating network transit risks.

### Threat 2: Unauthorized Data Access on Shared Devices

- **Risk:** Multiple users sharing a single mobile device or desktop computer could view each other's audit history or checklist configurations.
- **Mitigation:** LocalStorage data is stored inside the browser profile. Users are advised to use private browsing sessions or clear their browser cache in the settings menu when using shared devices.

### Threat 3: Local LLM Injection

- **Risk:** Extracted text could contain malicious prompts designed to manipulate the LLM's summary output.
  - **Mitigation:** GramLipi isolates prompts inside structured templates, instructing the local model to output strictly validated JSON.
- 

## 4. Mitigation Checklist

- **Image Downscaling:** Prior to writing reports, image files are processed on hidden canvases to minimize memory usage and keep base64 payloads compact.
- **Storage Clear Controls:** The settings panel includes a **Clear Data** option, allowing users to wipe all stored checklists, history, and achievements from localStorage.
- **Memory Release:** Background Web Workers (Tesseract workers) are terminated immediately after text extraction to free up device memory.